Diffusion in Depth

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{derive the forward process} \rightarrow \text{derive the loss} \rightarrow \text{sample} \rightarrow \text{sample faster} \rightarrow \text{guide} \rightarrow \text{work in a}$$

How to use this file. Part A derives everything step by step. Part B is a complete DDPM. Part C is practice with answers.

Step-up path

You need. Variational autoencoders and the diffusion basics.

1. Derive the forward process.
2. Derive the noise-prediction loss.
3. Sample with DDPM and DDIM.
4. Guide with a condition.
5. Compare latent diffusion and flow matching.

You can stop when you can derive and implement a diffusion model.

Contents

Part A Maths From the Ground Up

A1 The forward process

Each step adds a little Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\big).$$

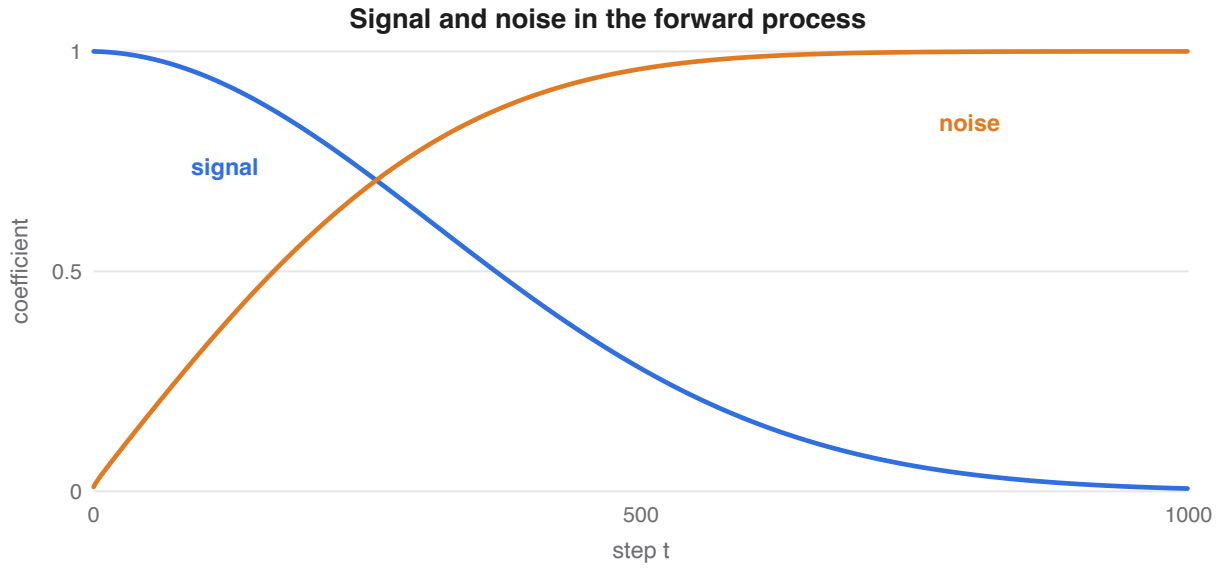
Write $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$.

Jumping straight to step t. Expand one step: $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\,\varepsilon$. Substituting the same form for x_{t-1} gives two independent Gaussians, whose variances add:

$$\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}.$$

By induction,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$



A2 The reverse process

If each β_t is small, the reverse step is also approximately Gaussian, so we learn its mean:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t),\ \sigma_t^2 I).$$

A3 The true posterior

Given x_0, the reverse step has a closed form. Bayes' rule with two Gaussians gives mean

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t,$$

and variance $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$.

A4 From the bound to predicting noise

The evidence lower bound becomes a sum of KL terms between Gaussians, each a scaled squared difference of means. Replace x_0 using A1, $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon)/\sqrt{\bar{\alpha}_t}$. Then

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon\right).$$

So predicting the mean is equivalent to predicting the noise, which gives the simple loss used in practice:

$$L = \mathbb{E}_{t, x_0, \varepsilon}\left\|\varepsilon - \varepsilon_\theta(x_t, t)\right\|^2.$$

A5 Sampling

Start from pure noise and repeat, from $t = T$ down to 1, with $z \sim \mathcal{N}(0, I)$ (and $z = 0$ on the last step):

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sigma_t z.$$

A6 The link with score matching

The **score** of $q(x_t \mid x_0)$ is $\nabla \log q = -\varepsilon/\sqrt{1-\bar{\alpha}_t}$. So a network that predicts noise is, up to a scale, estimating the score, which points towards higher probability. Diffusion and score-based models are the

same idea.

A7 Faster sampling: DDIM

Sampling with a thousand network evaluations is slow. **DDIM** uses the same trained network, with a deterministic update that skips steps. First estimate the clean image,

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta}{\sqrt{\bar{\alpha}_t}},$$

then move to an earlier noise level $s < t$:

$$x_s = \sqrt{\bar{\alpha}_s}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_s}\,\varepsilon_\theta.$$

Fifty steps are often enough.

A8 Classifier-free guidance

Train with conditioning c (such as a text embedding), randomly dropping it during training so the network also learns the unconditional case. At sampling time, extrapolate:

$$\hat{\varepsilon} = \varepsilon_\theta(x_t, \varnothing) + w\big(\varepsilon_\theta(x_t, c) - \varepsilon_\theta(x_t, \varnothing)\big).$$

$w = 1$ is plain conditional sampling, and larger w follows the condition more strongly, at the cost of diversity.

A9 Latent diffusion

High-resolution images make the U-Net costly. **Latent diffusion** first trains an autoencoder, then runs diffusion in its compressed space. A $512 \times 512 \times 3$ image compressed to $64 \times 64 \times 4$ has 48 times fewer numbers.

A10 Flow matching

Instead of a noising process, define a straight path between noise x_1 and data x_0:

$$x_t = (1-t)x_0 + t x_1, \qquad \frac{dx_t}{dt} = x_1 - x_0.$$

Train a velocity field to predict that direction, with $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$. To generate, integrate from noise back to data. Straighter paths need fewer steps.

Part B In Code

B1 A complete DDPM on two-dimensional data

```
import torch
from torch import nn

torch.manual_seed(0)
T = 200
beta = torch.linspace(1e-4, 0.02, T)
alpha = 1 - beta
abar = torch.cumprod(alpha, 0)


def data(n):
    """Two moons, centred."""
    t = torch.rand(n) * torch.pi
    top = torch.rand(n) < 0.5
    x = torch.stack([torch.cos(t), torch.sin(t)], 1)
    x[top] = torch.stack([1 - torch.cos(t[top]), 0.5 - torch.sin(t[top])], 1)
    return (x + 0.03 * torch.randn(n, 2) - torch.tensor([0.5, 0.25])) * 2


class Net(nn.Module):
    def __init__(self, w=128):
        super().__init__()
        self.t = nn.Embedding(T, w)
        self.net = nn.Sequential(nn.Linear(2 + w, w), nn.SiLU(), nn.Linear(w, w), nn.SiLU()
                                 nn.Linear(w, w), nn.SiLU(), nn.Linear(w, 2))

    def forward(self, x, t):
        return self.net(torch.cat([x, self.t(t)], 1))


model = Net()
```

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for step in range(4000):
    x0 = data(256)
    t = torch.randint(0, T, (256,))
    eps = torch.randn_like(x0)
    a = abar[t].unsqueeze(1)
    xt = a.sqrt() * x0 + (1 - a).sqrt() * eps
    loss = ((model(xt, t) - eps) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
    if step % 1000 == 0:
        print(step, round(loss.item(), 4))



@torch.no_grad()
def sample(n):
    x = torch.randn(n, 2)
    for i in reversed(range(T)):
        t = torch.full((n,), i)
        eps = model(x, t)
        x = (x - beta[i] / (1 - abar[i]).sqrt() * eps) / alpha[i].sqrt()
        if i > 0:
            x += beta[i].sqrt() * torch.randn_like(x)
    return x


print(sample(1000).std(0))
```

B2 DDIM sampling

```
@torch.no_grad()
def ddim(n, steps=20):
    x = torch.randn(n, 2)
    ts = torch.linspace(T - 1, 0, steps).long()
    for k, i in enumerate(ts):
        eps = model(x, torch.full((n,), int(i)))
        x0 = (x - (1 - abar[i]).sqrt() * eps) / abar[i].sqrt()
        a_next = abar[ts[k + 1]] if k + 1 < steps else torch.tensor(1.0)
        x = a_next.sqrt() * x0 + (1 - a_next).sqrt() * eps
    return x
```

B3 Classifier-free guidance

```
def guided(model, x, t, c, null, w):
    un = model(x, t, null)
    co = model(x, t, c)
    return un + w * (co - un)
```

Part C Practice

P1. $\bar{\alpha}_t = 0.25$. How much of the signal and the noise does x_t contain?

$\sqrt{0.25} = 0.5$ of x_0, and $\sqrt{0.75} = 0.866$ of the noise.

P2. Show why two Gaussian noise terms in the forward process combine into one.

Independent Gaussians add in variance, and the sum is still Gaussian: $\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t\alpha_{t-1}$.

P3. The network predicts $\varepsilon = 1$ with $\bar{\alpha}_t = 0.81$ and $x_t = 2$. Find $\hat{x}_0$.

$(2 - \sqrt{0.19})/0.9 = 1.738$.

P4. What does $w = 0$ do in classifier-free guidance?

It reduces to the unconditional prediction, ignoring the condition.

P5. How many times smaller is a $64 \times 64 \times 4$ latent than a $512 \times 512 \times 3$ image?

$786432/16384 = 48$.